

Индексы: B-tree, hash и bitmap

Table of contents

Индексы: B-tree, hash и bitmap	1
Что такое индекс	1
Почему индекс ускоряет чтение, но замедляет запись	2
B-tree как индекс по умолчанию	2
Hash-индекс	2
Bitmap-индекс	3
Как выбирать индекс	3
Связь с предыдущими лекциями	4

Индексы: B-tree, hash и bitmap

После лекций про B-дерево, LSM и физическое хранение строк полезно разделить два уровня разговора:

- **структура хранения и цена I/O:** immutable SSTable против mutable страниц;
- **тип индекса как способ доступа:** B-tree, hash, bitmap и их пригодность под разные запросы.

Эта лекция про второй уровень: что вообще делает индекс и почему у разных индексов разные сильные стороны.

Что такое индекс

Индекс в базе данных это отдельная структура, которая помогает найти нужные строки без полного сканирования таблицы.

Без индекса запрос вида

```
select * from users where email = 'a@b.c';
```

часто означает:

1. прочитать много страниц таблицы;
2. проверить условие для каждой строки;
3. выбрать подходящие строки.

С индексом база сначала идёт в компактную вспомогательную структуру, находит нужный ключ, а уже потом переходит к данным. Обычно индекс хранит что-то вроде:

search key → row id / page id / primary key

То есть индекс это не “таблица поменьше”, а **дополнительный путь доступа** к данным.

Почему индекс ускоряет чтение, но замедляет запись

Индекс экономит I/O на чтении, если запрос достаточно селективен: вместо чтения многих страниц таблицы можно прочитать несколько страниц индекса и затем перейти только к нужным строкам.

Но за это приходится платить:

- при `insert` надо добавить запись и в таблицу, и в индекс;
- при `update` индекс может потребовать перестройки, если изменился индексируемый столбец;
- при `delete` запись нужно убрать или пометить удалённой и в таблице, и в индексе;
- чем больше индексов на таблице, тем дороже каждая запись.

Поэтому индекс это всегда обмен: **ускоряем часть чтений ценой памяти, диска и более дорогих записей.**

B-tree как индекс по умолчанию

B-tree это самый универсальный индекс. Он хранит ключи в отсортированном порядке и поэтому умеет не только точечный поиск, но и работу с порядком ключей.

Что B-tree делает хорошо:

- поиск по равенству: `=`, `IN`;
- диапазоны: `<`, `<=`, `>`, `between`;
- префиксные условия;
- `order by`, `min`, `max`;
- предсказуемое поведение на смешанной OLTP-нагрузке.

Почему он универсален:

- ключи упорядочены;
- высота дерева мала из-за большого branching factor;
- чтение сводится к короткому пути по страницам;
- цена записи понятна: локальные `split/merge` страниц.

Если нет специальной причины выбрать что-то ещё, на практике чаще всего начинают именно с B-tree.

Hash-индекс

Hash-индекс пропускает ключ через хеш-функцию и кладёт его в корзину:

$$h(k) \rightarrow \text{bucket}$$

Сильная сторона hash-индекса:

- очень быстрый поиск **по точному равенству**;
- хорош, когда запросы почти всегда выглядят как `where id = ....`

Ограничения:

- он не хранит ключи в отсортированном порядке;
- не умеет диапазоны;
- не помогает для `order by`;
- плохо подходит для префиксного поиска.

Именно поэтому `hash`-индекс уже, чем `B-tree`. Он может быть быстрее в своей узкой задаче, но как индекс “по умолчанию” почти всегда проигрывает `B-tree` по универсальности.

Bitmap-индекс

Bitmap-индекс особенно полезен там, где столбец принимает **не очень много разных значений**. Для каждого значения хранится битовая маска: в каком наборе строк это значение встречается.

Пример для столбца `gender`:

- bitmap для `M`;
- bitmap для `F`.

Для столбца `status`:

- bitmap для `new`;
- bitmap для `paid`;
- bitmap для `cancelled`.

Почему это мощно в аналитике:

- фильтры можно быстро объединять через `AND`, `OR`, `NOT`;
- получается очень дешёвая работа с несколькими низкокардинальными предикатами;
- хорошо сжимается и хорошо подходит под DWH-нагрузки.

Почему bitmap плох в OLTP:

- частые обновления заставляют постоянно править битовые векторы;
- при высокой кардинальности bitmap разрастается и теряет смысл;
- индекс менее удобен для точечных обновлений и горячих записей.

Проще всего запомнить так: **bitmap любит аналитику и редкие изменения, B-tree любит универсальность, hash любит точное равенство.**

Как выбирать индекс

Тип запроса или нагрузки	Что обычно выбирать
<code>id = x</code> , точечный поиск	<code>B-tree</code> , иногда <code>hash</code>
<code>created_at between ...</code> , диапазоны	<code>B-tree</code>
<code>order by</code> , <code>min</code> , <code>max</code>	<code>B-tree</code>
несколько фильтров по низкокардинальным полям в аналитике	bitmap

Тип запроса или нагрузки	Что обычно выбирать
много записей, мало обновлений, OLAP/DWH	<code>bitmap</code> часто уместен
смешанная транзакционная нагрузка	<code>B-tree</code>

Связь с предыдущими лекциями

Теперь различие можно формулировать чище:

- **B-tree** и **LSM** это прежде всего разговор о том, как структура живёт на диске и как она платит за I/O;
- **heap** и **clustered** это разговор о том, где физически лежит полная строка таблицы;
- **B-tree, hash и bitmap** это разговор о типах индексов и о том, какие запросы они ускоряют;
- при этом B-tree одновременно попадает в обе категории: это и дисковая структура, и самый распространённый тип индекса.

Именно поэтому логично держать эти темы отдельно:

1. **В-дерево**: mutable страницы, branching factor, split/merge, чтение с диска.
2. **LSM-дерево**: immutable файлы, flush, compaction, write amplification.
3. **Хранение строк и индексы**: heap, clustered и переход от индекса к полной строке.
4. **Индексы**: что такое индекс вообще, и когда выбирать B-tree, hash или bitmap.